

- **Quais são as entidades (classes) que devem ser criadas?**

O ecossistema de streaming deve ser modelado separando os dados cadastrais, as regras comerciais de acesso e o catálogo de mídia:

1. **Usuario (Entidade):** Classe que representa o cliente assinante da plataforma, contendo suas informações básicas de identificação como nome e e-mail.
2. **PlanoAssinatura (Entidade/Objeto de Configuração):** Classe que carrega as regras e os limites comerciais de cada pacote (preço, qualidade de reprodução e o limite de telas simultâneas).
3. **Conteudo (Entidade):** Classe que representa os filmes ou séries do catálogo de mídia da plataforma, contendo o título, o formato do vídeo e a marcação se o arquivo é ou não de exibição exclusiva.
4. **Assinatura (Entidade / Raiz de Agregação):** A classe central que gerencia o vínculo financeiro e temporal entre o **Usuario** e o seu **PlanoAssinatura**. Ela controla se o serviço está ativo e calcula a data de expiração do acesso.
5. **TipoPlano (Enum):** Enumerador para padronizar as categorias comerciais: **Basico** e **Premium**.
6. **MetodoPagamento (Enum):** Enumerador para limitar os canais de recebimento mensal: **Cartao** e **Pix**.
7. **TipoConteudo (Enum):** Enumerador para classificar a mídia do catálogo: **Filme** e **Serie**.

- **Quais dessas entidades possuem ações (métodos) e quais são eles?**

Para manter os comportamentos protegidos e encapsulados dentro das classes corretas, os métodos foram mapeados da seguinte forma:

1. **Na classe PlanoAssinatura:**
 - **ConfigurarPlano():** Método interno executado no momento da criação do objeto. Ele atua como um distribuidor de regras, injetando os valores corretos de preço, telas e qualidade de imagem com base no enum escolhido.
 2. **Na classe Assinatura:**
 - **ProcessarPagamentoMensal(MetodoPagamento metodo):** Valida a transação financeira recorrente, altera o status da conta para ativa e adiciona exatamente trinta dias (um mês) à data da próxima cobrança.
 - **AssistirConteudo(Conteudo conteudo):** Método de validação de streaming que verifica as regras de negócio contra fraudes ou pirataria antes de liberar a transmissão do sinal de vídeo para o reproduzidor do usuário.
- **Há necessidade de criação de interface? Se sim, descreva os comportamentos que devem ser criados.**

Sim, em sistemas de streaming de grande escala que trabalham com transações automáticas e recorrentes (assinaturas), o uso de interfaces é fundamental para desacoplar a infraestrutura:

1. **ICobrancaRecorrente**: Essencial para gerenciar o agendamento de pagamentos mensais. Ela define o contrato para disparar as cobranças automáticas no cartão de crédito do cliente ou gerar o Pix todo mês sem que a plataforma dependa de um banco específico.
 - *Comportamentos*: `bool DebitarMensalidade(Usuario usuario, decimal valor);` e `void CancelarRecorrecencia(string assinaturaId);`
2. **IProvedorStreaming (Opcional)**: Abstrai o servidor de distribuição de vídeo (como AWS CloudFront ou Azure Media Services). A aplicação gerencia os planos, mas a entrega física do arquivo pesado de vídeo fica isolada atrás desta interface.
 - *Comportamento*: `string GerarLinkSeguro(Conteudo conteudo, string qualidade);`

• Quais são os relacionamentos entre as classes e entidades criadas?

As conexões entre os objetos refletem as dependências de acesso exigidas pela plataforma:

1. **Associação Simples (1 para 1)**: Uma `Assinatura` possui uma ligação direta com um `Usuario` e com um `PlanoAssinatura`. O usuário existe independentemente, mas a assinatura precisa apontar para um cliente e para um plano para ser válida.
2. **Dependência de Validação**: O método de transmissão da classe `Assinatura` recebe um objeto da classe `Conteudo` como parâmetro. Há uma relação de dependência temporária aqui, pois a assinatura precisa ler as propriedades do filme para saber se libera ou bloqueia o sinal.
3. **Composição por Atributos**: A classe `PlanoAssinatura` é composta pelo enum `TipoPlano`, a classe `Conteudo` carrega o enum `TipoConteudo`, e o fluxo de renovação da `Assinatura` consome o enum `MetodoPagamento`.

• Onde o desenvolvedor deve tratar exceções em caso de críticas no processo?

O tratamento de erros impede o uso indevido da plataforma e protege o faturamento do sistema:

1. **Na Camada de Domínio (Dentro da classe `Assinatura`)**: O desenvolvedor deve lançar exceções de operação inválida (`InvalidOperationException`) para cortar o fluxo do código imediatamente caso as regras de acesso ao catálogo sejam violadas. As duas principais críticas são:
 - Bloquear a transmissão e lançar erro se o usuário tentar invocar o método `AssistirConteudo` estando com a assinatura pausada ou com o pagamento atrasado (`Ativa == false`).

- Bloquear a transmissão e lançar erro se o usuário possuir o plano **Basico** e tentar assistir a um filme marcado com a propriedade **EhExclusivoPremium** configurada como verdadeira.
2. **Na Camada de Serviços / Infraestrutura (Gateway de Pagamento):** O desenvolvedor deve colocar blocos **try-catch** no motor de cobrança mensal. Se o cartão do cliente for recusado por falta de limite ou o banco do Pix estiver instável, a infraestrutura captura o erro de recusa, atualiza o status da **Assinatura** para "Inativa" no banco de dados para cortar o sinal de vídeo, salva o log da transação negada e envia uma notificação limpa na tela do usuário informando que houve um problema com a renovação da sua mensalidade.